

MERGE TWO SORTED LISTS

Company: Amazon

Round: Online Assessment

Year: 2019

Difficulty: Easy

Topic: Linked List, Two Pointers

Source: https://algo.monster/problems/merge_two_sorted_lists

Problem Description:

We are given two sorted linked lists.

Our task is to combine them into one sorted linked list.

For example:

$l1 = 1 \rightarrow 2 \rightarrow 4$

$l2 = 1 \rightarrow 3 \rightarrow 4$

After merging them, we should get:

$1 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 4$

The important thing is that the final list must also be sorted.

We don't need to create completely new nodes. We can simply connect the existing nodes from the two lists in the correct order.

Input Format:

We are given two sorted linked lists: $l1$ and $l2$

Both lists are already sorted in non-decreasing order. That means the values can stay the same or increase, but they cannot decrease.

Output Format:

Return one linked list containing all the nodes from $l1$ and $l2$, arranged in sorted order.

Sample Input:

$l1 = [1, 2, 4]$

$l2 = [1, 3, 4]$

Sample Output:

$[1, 1, 2, 3, 4, 4]$

Explanation:

What is a Linked List?

A linked list is a collection of nodes.

Each node contains:

value + next

For example:

$1 \rightarrow 2 \rightarrow 4 \rightarrow \text{null}$

Let's take the first example:

$l1 = 1 \rightarrow 2 \rightarrow 4$

$l2 = 1 \rightarrow 3 \rightarrow 4$

We compare the first values of both lists.

1 and 1

Both are equal, so we take one of them.

Now we compare:

2 and 1

1 is smaller, so we take it.

Then:

2 and 3

2 is smaller, so we take 2.

Next:

4 and 3

3 is smaller, so we take 3.

Then:

4 and 4

We take one 4.

Finally, the remaining 4 is added.

The result is:

$1 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 4$

Approach to Solve This Problem:

The easiest way to solve this is to use two pointers.

We keep:

- One pointer at the current node of l1
- One pointer at the current node of l2

Then we compare their values.

Step 1: Compare the two nodes

Suppose:

$l1 = 1$
 $l2 = 3$

Since 1 is smaller, we put 1 into the result.

Then we move the l1 pointer forward.

Step 2: Continue comparing

Now suppose:

$l1 = 2$
 $l2 = 3$

Again, 2 is smaller.

So we add 2 and move l1 forward.

We continue doing this until one of the lists becomes empty.

Step 3: Add the remaining list

Suppose l1 becomes empty but l2 still has:

$4 \rightarrow 5 \rightarrow 6$

We don't need to compare anything anymore because l2 is already sorted.

We can simply attach the remaining part of l2.

The same applies if l2 becomes empty first.

Why Do We Use Two Pointers?

Because both lists are already sorted.

We don't need to look at every element and sort everything again.

We simply compare the current elements and choose the smaller one.

This makes the solution faster and easier.

Java Code:

```
1  import java.util.*;
2
3  class ListNode {
4      int val;
5      ListNode next;
6
7      ListNode(int val) {
8          this.val = val;
9          this.next = null;
10     }
11 }
12
13 public class Main {
14
15     public static ListNode mergeTwoLists(ListNode l1, ListNode l2) {
16
17         // Dummy node helps us easily build the result
18         ListNode dummy = new ListNode(val: 0);
19         ListNode current = dummy;
20
21         // Compare both lists while both have nodes
22         while (l1 != null && l2 != null) {
23
24             if (l1.val <= l2.val) {
25                 current.next = l1;
26                 l1 = l1.next;
27             } else {
28                 current.next = l2;
29                 l2 = l2.next;
30             }
31
32             current = current.next;
33         }
34
35         // Add whatever is left
36         if (l1 != null) {
37             current.next = l1;
38         } else {
39             current.next = l2;
```

```
// Compare both lists while both have nodes
while (l1 != null && l2 != null) {

    if (l1.val <= l2.val) {
        current.next = l1;
        l1 = l1.next;
    } else {
        current.next = l2;
        l2 = l2.next;
    }

    current = current.next;
}

// Add whatever is left
if (l1 != null) {
    current.next = l1;
} else {
    current.next = l2;
}

return dummy.next;
}

// Creates a linked list from an array
public static ListNode createList(int[] arr) {

    ListNode dummy = new ListNode(val: 0);
    ListNode current = dummy;

    for (int value : arr) {
        current.next = new ListNode(value);
        current = current.next;
    }

    return dummy.next;
}
```

```
// Prints the linked list
public static void printList(ListNode head) {

    System.out.print(s: "[");

    while (head != null) {

        System.out.print(head.val);

        if (head.next != null) {
            System.out.print(s: ",");
        }

        head = head.next;
    }

    System.out.println(x: "]");
}
```

Run | Debug

```
public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    // Number of nodes in first list
    int n1 = sc.nextInt();

    int[] arr1 = new int[n1];

    for (int i = 0; i < n1; i++) {
        arr1[i] = sc.nextInt();
    }

    // Number of nodes in second list
    int n2 = sc.nextInt();

    int[] arr2 = new int[n2];
}
```

```
for (int i = 0; i < n2; i++) {  
    arr2[i] = sc.nextInt();  
}  
  
// Create the two linked lists  
ListNode l1 = createList(arr1);  
ListNode l2 = createList(arr2);  
  
// Merge them  
ListNode result = mergeTwoLists(l1, l2);  
  
// Print the result  
printList(result);  
  
sc.close();  
}
```

Solution:

The solution is based on a very simple idea:

Compare the current nodes of both sorted lists and always choose the smaller value. We continue until one list is empty.

Then we attach the remaining part of the other list.

For:

$l1 = 1 \rightarrow 2 \rightarrow 4$

$l2 = 1 \rightarrow 3 \rightarrow 4$

we get:

$1 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 4$

This approach is efficient because we don't need to sort the values again.

Understanding the Important Parts of Code:

1. Two pointers

The two pointers are:

$l1$

$l2$

They point to the current nodes we are comparing.

2. Compare the values

if ($l1.val \leq l2.val$)

If the value in $l1$ is smaller, we add that node to the result.

Otherwise, we add the node from $l2$.

3. Move the pointer

After using a node, we move that list's pointer forward:

$l1 = l1.next$;

or:

$l2 = l2.next$;

4. Move the result pointer

$current = current.next$;

This moves $current$ to the node we just added.

5. Add the remaining nodes

When one list becomes empty:

```
if ( $l1 \neq null$ ) {  
     $current.next = l1$ ;  
} else {  
     $current.next = l2$ ;  
}
```

We simply attach the remaining list because it is already sorted.

Complexity Analysis:

Let:

- n = number of nodes in $l1$
- m = number of nodes in $l2$

Time Complexity: $O(n + m)$

We go through each node of both linked lists only once while merging them.

Space Complexity: $O(1)$

We only use a few pointers (l1, l2, and current). We reuse the existing nodes instead of creating a new node for every element.